

Лекция 10: Введение в Firebase и Firestore.



Хекслет Колледж

Цель занятия:

Сформировать у студентов понимание Firebase как платформы Backend-as-a-Service и приобрести практические навыки создания проекта, настройки Firestore и выполнения базовых операций с данными.

Учебные вопросы:

1. Что такое Firebase и зачем он нужен фронтенд-разработчику?
2. Что такое Firestore?
3. Как устроена модель данных в Firestore ?
4. Как создать проект и базу данных в Firebase Console?
5. Как подключить Firebase к веб-приложению на JavaScript?

1. Что такое Firebase и зачем он нужен фронтенд-разработчику?

Ключевая проблема фронтенд-разработчика

Ситуация: Вы сделали красивый интерфейс интернет-магазина на React/Vue, но:

- Некуда сохранять товары и заказы
- Негде регистрировать пользователей
- Негде хранить фотографии товаров
- Нет сервера для обработки данных

Традиционное решение: Изучать бэкенд (Node.js, Python, базы данных), настраивать сервер, писать API...

Проблема: Это требует месяцев обучения, денег на хостинг и времени на поддержку.

Что такое Firebase?

Firebase — это Backend-as-a-Service (BaaS) платформа от Google.

Простая аналогия:

Если веб-разработка — это строительство дома:

- Frontend (HTML/CSS/JS) — это дизайн и отделка комнат
- Традиционный бэкенд — это фундамент, коммуникации, которые нужно строить самому
- Firebase — это готовый "фундамент с коммуникациями", который можно арендовать

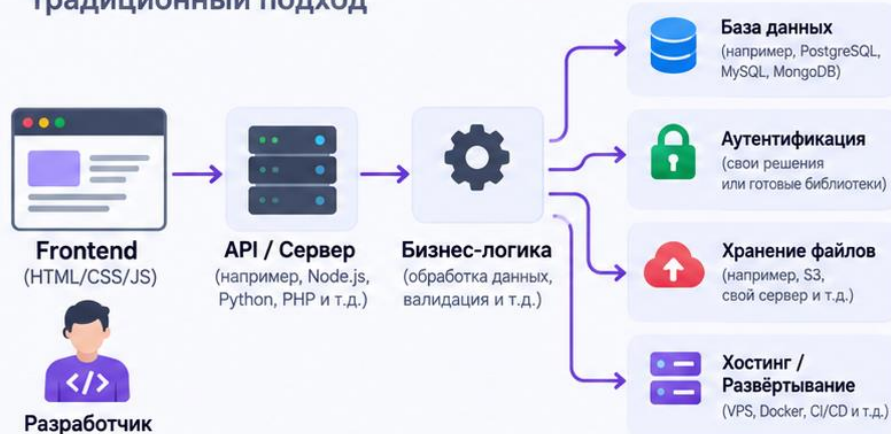
Firebase = готовый бэкенд в облаке, который настраивается через консоль и используется из JavaScript.

Frontend без Firebase / Frontend с Firebase

Один результат — разные пути. Выбирай подход под свои задачи!

Без Firebase

Традиционный подход



Нужно писать сервер



Нужно настраивать БД



Больше времени на запуск

Итог: больше настроек и больше кода

С Firebase

BaaS-подход



Без собственного сервера



Быстрый старт



Готовые сервисы

Итог: быстрее собрать рабочий прототип



Настройка: **долго** / **быстро**



Инфраструктура: **своя** / **готовая**



Старт проекта: **сложнее** / **проще**

Зачем именно фронтенд-разработчику?

1. Можно делать fullstack-приложения без изучения бэкенда:

- Вы уже знаете JavaScript
- Firebase имеет JavaScript SDK

Результат: Можете создать полностью рабочее приложение (интернет-магазин, чат, блог) самостоятельно

2. Быстрый старт проекта:

- Создание проекта: 5 минут вместо 5 дней
- Готовые решения: аутентификация, база данных, хостинг в одном месте
- Бесплатный тариф: достаточно для 90% учебных и стартап-проектов

3. Портфолио и практика:

- Проблема студента: "В моём портфолио только статические сайты"
- Решение с Firebase: Можете добавить реальные проекты с:
 - Регистрацией пользователей
 - Базами данных
 - Загрузкой файлов
 - Real-time обновлениями

Экосистема **Firebase**

Какие сервисы входят в платформу и для чего они нужны

Основные для старта

Минимальный набор для создания приложения



Firestore

NoSQL база данных



Authentication

регистрация и вход



Storage

хранение файлов



Hosting

публикация сайта



Firebase

БaaS-платформа
от Google

Дополнительные возможности

Расширяют возможности вашего приложения



Cloud Functions

серверная логика



Cloud Messaging

push-уведомления



Analytics

аналитика пользователей



Быстрый запуск



Готовые сервисы



Меньше серверной рутины

Ключевые сервисы Firebase для веб-разработчика:

Сервис	За что отвечает	Аналог в традиционной разработке
Firestore	База данных для товаров и заказов	MongoDB + сервер Node.js
Authentication	Регистрация и вход пользователей	Passport.js + сессии/куки
Storage	Хранение фотографий товаров	Multer + облачное хранилище
Hosting	Размещение готового сайта	Nginx/Apache на VPS

Полезные дополнительные:

Сервис	Для чего нужен
Cloud Functions	Серверный код (например, отправка email о заказе)
Cloud Messaging	Push-уведомления для мобильных приложений
Analytics	Аналитика пользовательского поведения

Real-кейс: Интернет-магазин на Firebase

Без Firebase:

- Арендовать сервер (от \$10/месяц)
- Установить Node.js, настроить Express
- Настроить базу данных (PostgreSQL/MongoDB)
- Написать API для товаров, заказов, пользователей
- Настроить аутентификацию
- Настроить загрузку файлов
- Развернуть фронтенд

Итого: ~2-4 недели работы, нужен опыт бэкенд-разработки

С Firebase:

- Создать проект в консоли (5 мин)
- Включить Firestore, Authentication, Storage
- Подключить SDK к фронтенду (10 мин)
- Начать писать логику приложения

Итого: ~1 день до первого прототипа, нужен только JavaScript

Когда НЕ использовать Firebase?

Не подходит, если:

- Сложная бизнес-логика на сервере (банковские операции, сложные расчёты)
- Жёсткие требования к данным (юридические, медицинские данные с особыми требованиями)
- Очень большие объёмы данных (может стать дорого)
- Нужен полный контроль над инфраструктурой

Идеально подходит для:

- Стартапов и прототипов
- Учебных проектов
- Приложений со средним трафиком
- Команд без бэкенд-разработчиков
- Real-time приложений (чаты, уведомления)

Когда Firebase подходит, а когда — нет?

Firebase — мощный инструмент, но не универсальное решение.
Важно понимать, для каких задач он подходит лучше всего.



Firebase подходит, если:

Нужно быстро, удобно и без серверной инфраструктуры



Быстрый запуск проекта

Нужно быстро проверить идею или сделать MVP



Веб- и мобильные приложения

Особенно с клиентской разработкой (JavaScript, Flutter и др.)



Малые и средние проекты

Стартапы, учебные проекты, личные приложения



Нужны данные в реальном времени

Чаты, уведомления, совместная работа



Нужны готовые облачные сервисы

База данных, аутентификация, хранилище, хостинг и т.д.



Ограниченный бюджет

Есть бесплатный тариф (Spark) для небольших проектов

Примеры:

- Учебные проекты и дипломные работы
- MVP стартапов
- Чаты, социальные сети, приложения с уведомлениями
- Прототипы и личные проекты
- Мобильные приложения на Flutter/React Native



Вывод: Firebase — отличный выбор для большинства учебных, личных и коммерческих проектов, но всегда оценивайте требования вашего проекта и выбирайте подходящий инструмент.



Firebase не подходит, если:

Нужен полный контроль, сложная логика или особые требования



Сложная серверная логика

Нужна сложная бизнес-логика, которая лучше реализуется на собственном backend (например, микросервисы)



Полный контроль над инфраструктурой

Требуется гибкая настройка серверов, сетей, баз данных



Работа с большими объёмами данных

Проекты с очень большими нагрузками и сложными запросами (может быть дорого)



Особые требования к безопасности и соответствию

Например, банковские системы, государственные проекты, строгие регуляторные требования (GDPR, HIPAA и др.)



Использование специфических технологий

Нужны технологии, которых нет в экосистеме Firebase



Чувствительность к стоимости при масштабировании

При большом росте аудитории стоимость может стать высокой

Примеры:

- Крупные корпоративные системы
- Финансовые и государственные проекты
- Приложения с очень сложной бизнес-логикой
- Системы с высокими требованиями к производительности
- Проекты, где критичен полный контроль над инфраструктурой

Нет универсальных решений.
Есть подходящие решения
для конкретных задач.

Firebase в цифрах

Бесплатный тариф (Spark Plan):

- Firestore: 1 ГБ хранилища, 50 тыс. чтений/день
- Storage: 5 ГБ, 1 ГБ трафика/день
- Hosting: 10 ГБ, 360 МБ/день трафик
- Authentication: 10 тыс. активных пользователей
- Достаточно для: товаров с описаниями
- 1000 заказов
- 5000 пользователей
- Ежедневная аудитория до 1000 человек

Альтернативы Firebase

Другие BaaS-платформы:

- Supabase — "Open Source Firebase" на PostgreSQL
- AWS Amplify — от Amazon, больше настроек
- Back4App — на основе Parse Server

Но Firebase:

- Самый популярный (больше туториалов, решений проблем)
- Лучшая документация
- Плавный переход на платные тарифы
- Интеграция с другими сервисами Google

Выводы:

Firebase для студента-фронтендера это:

- Возможность создать полноценное приложение сразу
- Портфолио с реальными проектами
- Понимание работы с данными без углубления в бэкенд
- Быстрый результат для мотивации
- Бесплатный старт для обучения

Главный принцип:

"Не изучайте бэкенд с нуля. Используйте готовый бэкенд (Firebase) и сосредоточьтесь на том, что вы уже умеете - фронтенд-разработке."

2. Что такое Firestore?

Firestore — это облачная NoSQL база данных от Google, которая позволяет фронтенд-разработчикам хранить и синхронизировать данные прямо из JavaScript-кода без написания серверной части.

Ключевые особенности Firestore

1. Документная модель данных (как папки и файлы)
2. Работает прямо из браузера (без сервера)
3. Real-time обновления (самая крутая фишка)
4. Офлайн-режим, работа без интернета, за счет кэширования

Ограничения и важные моменты

Что можно:

- ✓ Хранить структурированные данные (товары, пользователи, заказы)
- ✓ Делать сложные запросы (фильтры, сортировка, пагинация)
- ✓ Работать офлайн
- ✓ Настроить права доступа (покупатель ≠ админ)
- ✓ Автоматическое масштабирование

Что нельзя (или сложно):

- ✗ Сложные SQL-джойны
- ✗ Полнотекстовый поиск
- ✗ Сложные агрегации
- ✗ Хранить файлы >1 МБ

3. Как устроена модель данных в Firestore ?

Ключевая метафора: Библиотека

Firestore — это цифровая библиотека:

- Библиотека = База данных Firestore
- Стеллажи = Коллекции
- Книги = Документы
- Главы в книге = Поля документа
- Папка с рецензиями внутри книги = Подколлекция

Модель данных Firestore

Как устроены коллекции, документы, поля и подколлекции

Иерархия

От общего к частному



Firestore проект
контейнер приложения



База данных Firestore
хранит коллекции



Коллекция (collection)
группа однотипных документов



Документ (document)
одна запись с данными



Поля (fields)
пары ключ: значение



Подколлекция (subcollection)
вложенная коллекция
внутри документа

Как это выглядит

Пример структуры интернет-магазина



Firestore проект

База данных Firestore

products (коллекция)

iphone_15 (документ)

{ name: "iPhone 15"

price: 549000

inStock: true

specs (map)

reviews (подколлекция)

review_1 (документ)

{ rating: 5

text: "Отличный телефон"

users (коллекция)

user_123 (документ)

{ name: "Анна"

email: "anna@example.com"

orders (подколлекция)

...

Запомнить

Главные принципы работы с Firestore



Коллекция содержит
документы



Документ содержит
поля



Документ может иметь
подколлекции



У коллекции нет
собственной схемы

Аналогия

Простое сравнение для понимания



Коллекция = папка / таблица



Документ = файл / строка



Поле = ячейка с данными



Гибкая структура

Легко адаптируется под ваши задачи



NoSQL подход

Храните любые типы данных



Удобно для веб-приложений

Масштабируется вместе с вашим проектом

Иерархия: от общего к частному



Firestore Проект

- └─  База данных Firestore (по умолчанию одна)
 - └─  Коллекция (collection) → например, "products"
 - └─  Документ (document) → например, "iphone-15"
 - └─  Поле (field) → "name": "iPhone 15"
 - └─  Поле → "price": 999
 - └─  Поле → "inStock": true
 - └─  Подколлекция (subcollection) → "reviews"
 - └─  Документ → "review1"
 - └─  Поле → "text": "Отличный телефон!"
 - └─  Поле → "rating": 5

Коллекция (Collection) — как таблица в SQL

Что такое:

- Группа однотипных документов. Аналог таблицы в SQL или папки в компьютере.

Примеры для магазина:

- products — все товары
- users — все пользователи
- orders — все заказы
- categories — категории товаров

Особенности:

- ✓ Не имеет полей — только содержит документы
- ✓ Нет схемы — документы внутри могут иметь разную структуру
- ✓ Автоматически не создаётся — создаётся при добавлении первого документа

Документ (Document) — как строка в таблице

Что такое:

- Отдельная запись в коллекции. Содержит данные в формате ключ-значение.

ID документа:

- Автоматически (если не указать): Lp2r4sT8qK1n (рандомный)
- Вручную (если важно): iphone-15-pro, user-123

Ограничения документа:

- Максимум 1 МБ данных
- Максимум 20 000 полей (но лучше < 100)
- Нельзя хранить файлы > 10 КБ (используем Storage)

Поле (Field) — ячейка с данными

Типы данных в Firestore:

Тип	Пример	Зачем нужно
String	"iPhone 15"	Текст
Number	999, 99.99	Цены, количества
Boolean	true/false	Флаги (в наличии/нет)
Timestamp	Timestamp.now()	Дата создания заказа
Array	["red", "blue"]	Цвета товара, теги
Map/Object	{memory: "256GB"}	Вложенные данные
Null	null	Отсутствие значения
Reference	db.doc("users/abc123")	Ссылка на другой документ
GeoPoint	new GeoPoint(55.75, 37.61)	Координаты доставки

Пример:

```
// Товар в магазине
const product = {
  name: "iPhone 15",           // String
  price: 999.99,               // Number (деньги)
  inStock: true,               // Boolean
  createdAt: firebase.firestore.FieldValue.serverTimestamp(), // Авто-дата
  colors: ["black", "white", "gold"], // Array вариантов
  specs: {                     // Map для сложных данных
    memory: "256GB",
    display: "6.1 inches"
  },
  categoryRef: db.doc("categories/smartphones"), // Reference
  location: new firebase.firestore.GeoPoint(55.75, 37.61) // Для доставки
};
```

Подколлекция (Subcollection) — документ внутри документа

Что такое:

- Коллекция, которая принадлежит конкретному документу.

```
users (коллекция)
└─ user_123 (документ пользователя)
    ├── name: "Анна"
    ├── email: "anna@mail.ru"
    └─ orders (подколлекция заказов этого пользователя)
        └─ order_456 (документ заказа)
            ├── total: 1999
            └─ status: "delivered"
```

4. Как создать проект и базу данных в Firebase Console?

Что нужно сделать:

- Создать аккаунт Google (если нет)
- Зайти в Firebase Console
- Создать новый проект
- Добавить Firestore базу данных
- Настроить в тестовом режиме
- Добавить первый товар



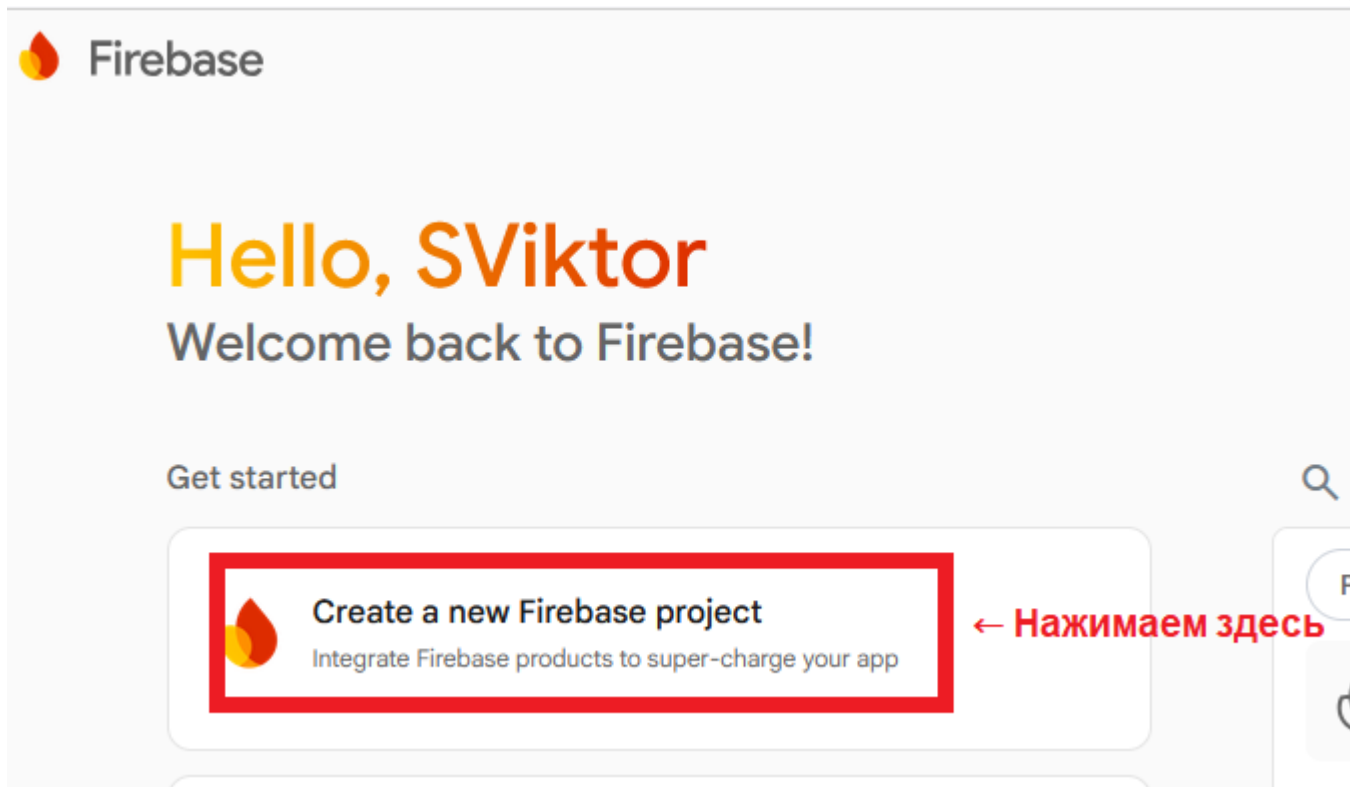
Конечный результат:

- Готовая база данных с первым товаром для нашего интернет-магазина.

Шаг 1: Вход в Firebase Console

- Откройте браузер (Chrome, Firefox, Edge)
- В адресной строке введите:
console.firebase.google.com
- Войдите под своим Google-аккаунтом

Шаг 2: Создание проекта



Название проекта

× Create a project

Let's start with a name for
your project[?]

Project name

my-test-project

Create a project



Get campaign insights and recommendations to engage your users through Firebase Cloud Messaging



Generate schema and explore your data using natural language in Firebase Data Connect



Enable Gemini in Firebase

Recommended

Disclaimers:

Gemini in Firebase is still under development and may give inaccurate information. Test any code it generates before using it in your apps. Gemini can interact with your Firebase data and tailor response to your app, and it may use your prompts or data for training. [Learn more about how we train our models](#) Use of Gemini in Firebase is subject to [Google Terms of Service](#) and the [Generative AI Prohibited Use Policy](#)

[Previous](#)

[Continue](#)

Google Analytics (пропускаем)

Create a project

Remote Config, A/B Testing, and Cloud Functions.

Google Analytics enables:

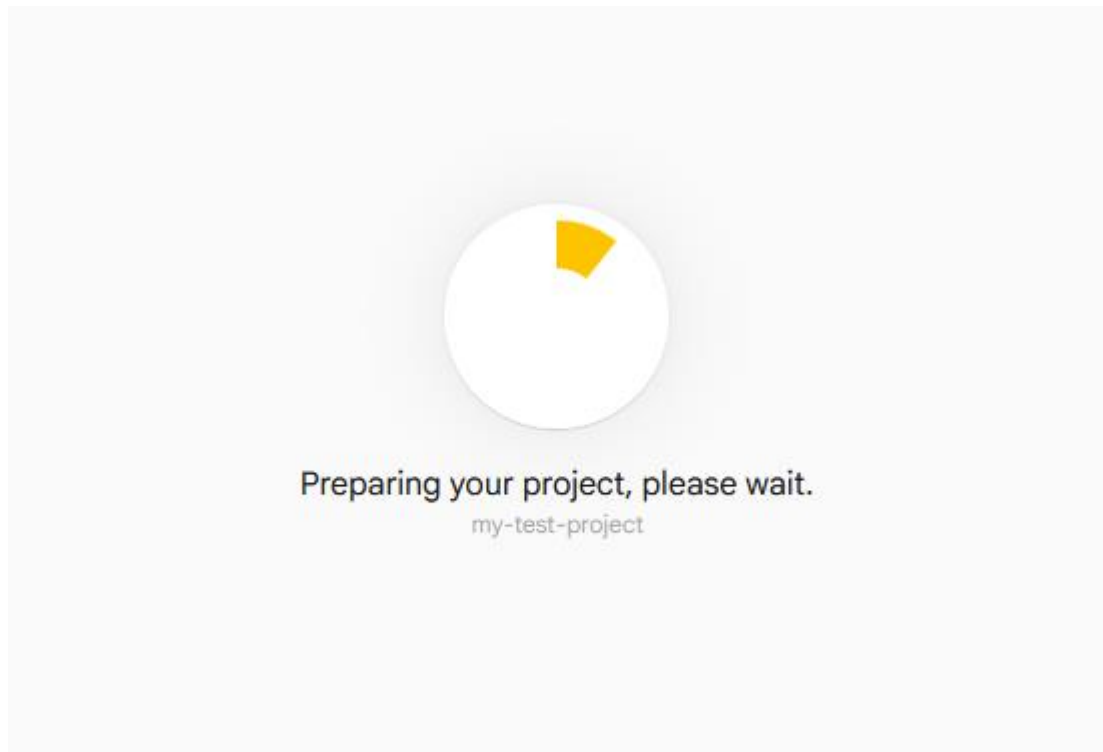
- ☒ A/B testing ?
- ☒ User segmentation & targeting across Firebase products ?
- ☒ Breadcrumbs logs in Crashlytics ?
- ☒ Event-based Cloud Functions triggers ?
- ☒ Free unlimited reporting ?

☒ Enable Google Analytics for this project
Recommended

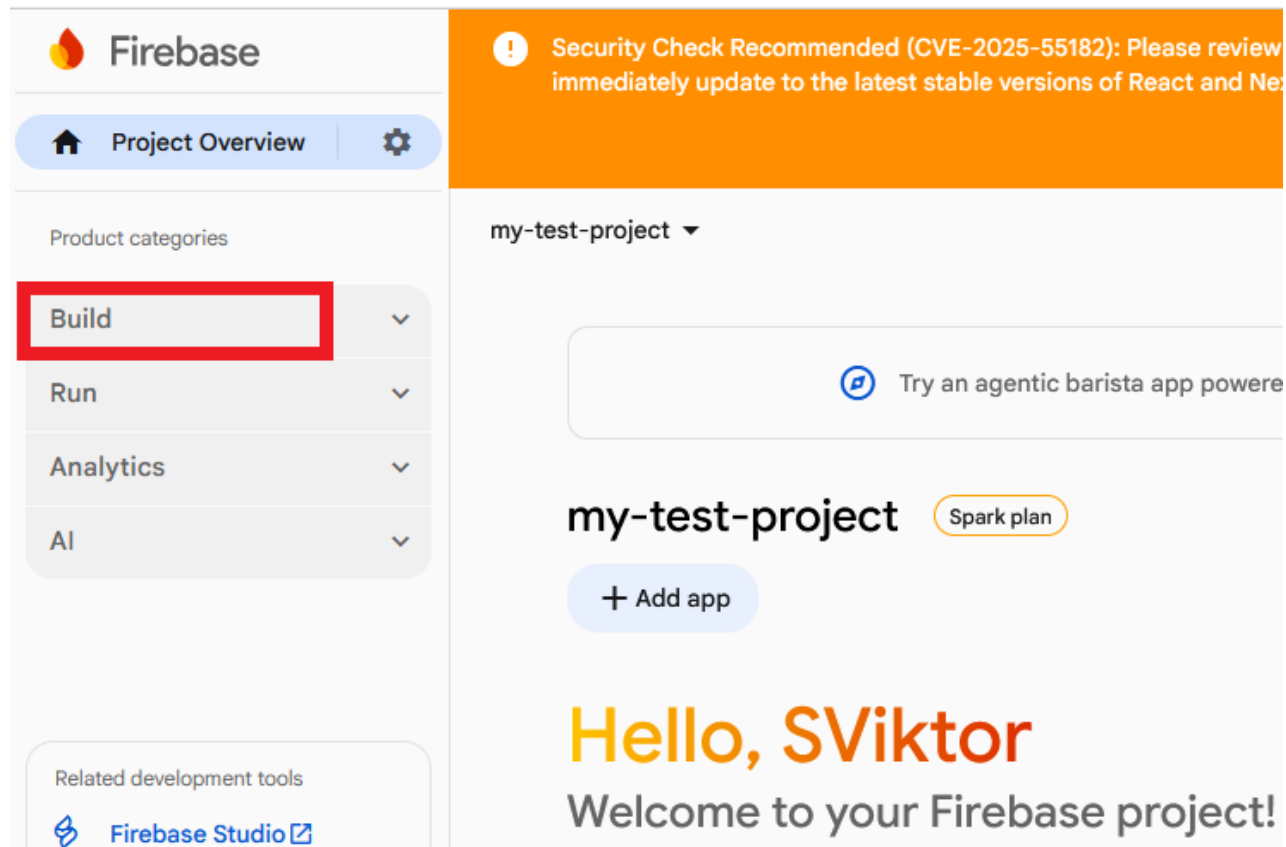
[Previous](#)

Create project

Ждём создания



Шаг 3: Добавление базы данных Firestore



 **Firebase**

 Project Overview 

Product categories

Build 

 App Check

 App Hosting

 Authentication

 Data Connect

 Extensions

 **Firestore Database**

 Functions

 Hosting

 Machine Learning

 **Security Check Recommended (CVE-2025-55182):** Please re...
immediately update to the latest stable versions of React and

my-test-project ▼

 Try an agentic barista app po

my-test-project **Spark plan**

 Add app

Hello, SViktor

Welcome to your Firebase project



Project shortcuts

**Firestore Database**

Product categories

Build ▾

Run ▾

Analytics ▾

AI ▾

Related development tools

[4](#) [5](#) [6](#) [7](#) [8](#)

Cloud Firestore

Realtime updates, powerful queries,
automatic scaling, and MongoDB
compatibility

Create database**Ask Gemini**

× Create a database

1 Select edition



Standard edition

Simple query engine with automatic indexing

Supports Core operations



Enterprise edition

Advanced query engine indexing.

Supports Core, Pipelir operations

Not sure which edition is right for you? [Compare editions](#) 

Next

Create a database

2 Database ID & location

Database ID

(default)

Location

asia-south1 (Mumbai)

 Your location setting is where your Cloud Firestore data will be stored



After you set this location, you cannot change it later

Next

Начинаем в тестовом режиме (ДЛЯ УЧЕБЫ)

Create a database

After you define your data structure, you will need to write rules to secure your data. [Learn more](#)

☐ Start in **production mode**

Your data is private by default. Client read/write access will only be granted as specified by your security rules.

☒ Start in **test mode**

Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if
        request.time < timestamp.date(
    }
  }
}
```



The default security rules for test mode allow a your database reference to view, edit and delete your database for the next 30 days

Cancel

Create

Шаг 4: Проверка созданной базы

Cloud Firestore >

Database

[Add database](#) [Ask Gemini about the core concepts to use Firestore](#)

[Data](#) [Rules](#) [Indexes](#) [Disaster Recovery](#) [Usage](#) | [Extensions](#)

 Protect your Cloud Firestore resources from abuse, such as billing fraud or phishing [Configure](#)



 (default)	
+ Start collection	

Шаг 5: Добавление первого товара

Start a collection

1 Give the collection an ID

2 Add its first document

Parent path

/

Collection ID ?

products

Cancel

Next

Добавляем первый документ

Auto-ID

Required

Field	Type	
name	string	⊖
String		
<u>iPhone 15</u>		

Field	Type	Value	
price	number	549000	⊖

Field	Type	
category	string	⊖
String		
<u>smartphones</u>		

Field	Type	Value	
inStock	boolean	true	⊖

Результат:

🏠 > products > 6pSjcsO6qqoit...

🏠 (default)	📁 products	📄 6pSjcsO6qqoit4QrEEBn
+ Start collection	+ Add document	+ Start collection
products >	6pSjcsO6qqoit4QrEEBn >	+ Add field category: "smartphones" inStock: true name: "iPhone 15" price: 549000

Добавляем ещё товары:

MacBook Pro, цена: 1999, категория: laptops

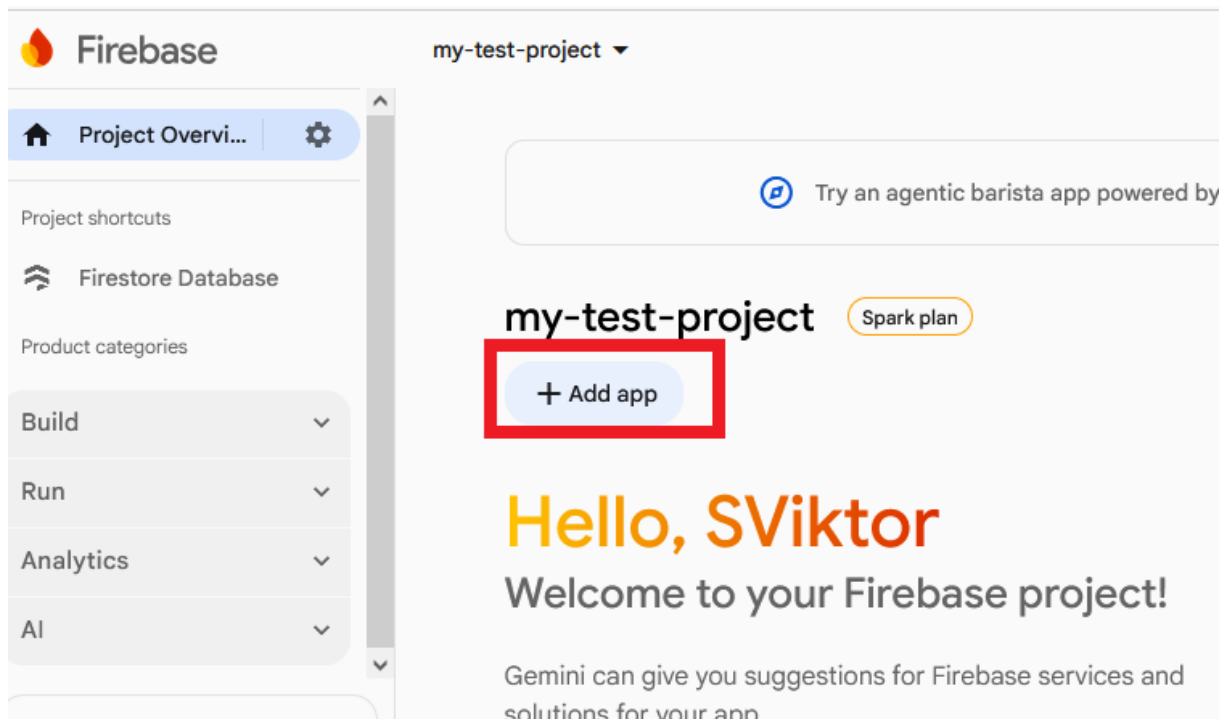
AirPods Pro, цена: 249, категория: audio

Результат:

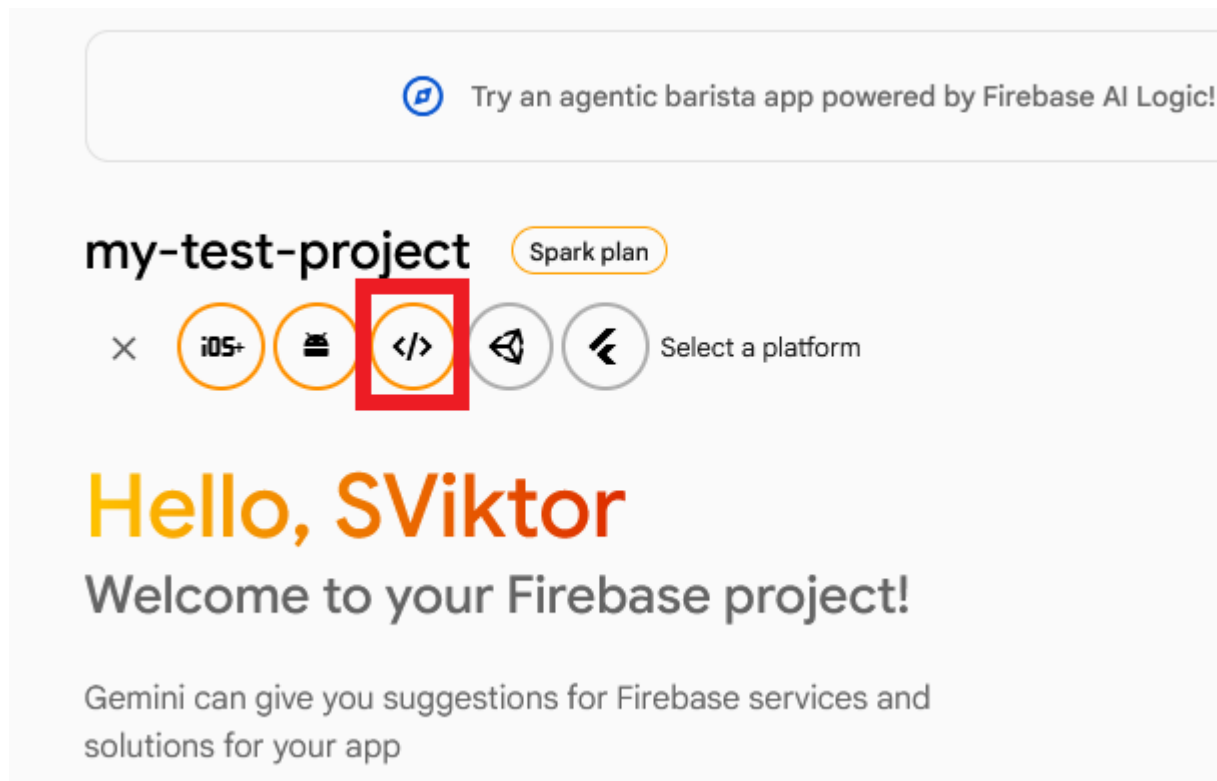
 > products > NnaZlhSGHRt2...		
 (default)	 products  	 NnaZlhSGHRt2CvgbDv2b
+ Start collection	+ Add document	+ Start collection
products >	6pSjcs06qqoit4QrEEBn	+ Add field
	NnaZlhSGHRt2CvgbDv2b >	category: "audio"
	idhpPQMYK5hYUWRUOB0b	inStock: true
		name: "AirPods Pro"
		price: 99000

Шаг 7: Получение конфигурации для кода

Возвращаемся на главную => Выбираем проект => добавить приложение



Выбираем веб-приложение:



Вводим название приложения

× Add Firebase to your web app

1

Register app

App nickname ?

Shop Frontend

☐ Also set up **Firebase Hosting** for this app. [Learn more](#) ↗

Hosting can also be set up later. There is no cost to get started anytime.

Register app

Копируем и сохраняем код в отдельный файл или блокнот

2 Add Firebase SDK

☐ Use npm

☒ Use a `<script>` tag

If you don't use build tools, use this option to add and use the Firebase JS SDK. Use this option to get started, but it's not recommended for production apps. [Learn more](#).

Copy and paste these scripts into the bottom of your `<body>` tag, but before you use any Firebase services:

```
<script type="module">
  // Import the functions you need from the SDKs you need
  import { initializeApp } from "https://www.gstatic.com/firebasejs/12.8.0/firebase-app.js";
  // TODO: Add SDKs for Firebase products that you want to use
  // https://firebase.google.com/docs/web/setup#available-libraries

  // Your web app's Firebase configuration
  const firebaseConfig = {
    apiKey: "AIzaSyA...",
    authDomain: "my-test-project-e0b7a.firebaseio.com",
    projectId: "my-test-project-e0b7a",
    storageBucket: "my-test-project-e0b7a.firebaseio.com",
    messagingSenderId: "962855943660",
    appId: "1:962855943660:web:278bfe37376e6b046154c9"
  };

  // Initialize Firebase
  const app = initializeApp(firebaseConfig);
</script>
```

Завершаем настройку

```
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyB...",
  authDomain: "my-test-project-e0b7a.firebaseio.com",
  projectId: "my-test-project-e0b7a",
  storageBucket: "my-test-project-e0b7a.firebaseio.com",
  messagingSenderId: "962855943660",
  appId: "1:962855943660:web:278bfe37376e6b046154c9"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
</script>
```

Are you using npm and a bundler like webpack or Rollup? Check out the [modular SDK](#).

Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)

Continue to console

Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)

Измените правила (для разработки)

1. Firebase Console → Ваш проект
2. Слева в меню: ► Build (Сборка)
3. Развернуть: Firestore Database
4. Появится подменю:
 - └── Data (Данные)
 - └── Rules (Правила) ← ВОТ ОНО!
 - └── Indexes (Индексы)

my-test-project ▾

Cloud Firestore >

Database

Add database

◆ Ask Gemini about the core concepts to use Firestore

Data

Rules

Indexes

Disaster Recovery

Usage

🔌 Extensions



Guard your data with rules that define who has access to it and how it is structured

Заменить

```
1 rules_version = '2';
2
3 service cloud.firestore {
4     match /databases/{database}/documents {
5         match /{document=**} {
6             allow read, write: if false;
7         }
8     }
9 }
```

Замените существующие правила на:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if true; // Разрешаем ВСЁ
    }
  }
}
```

Никогда не использовать в реальном проекте!!!



Security Rules: плохо / лучше

Почему нельзя оставлять открытые правила и как сделать безопаснее

Плохо

Открытый доступ ко всей базе

Firestore Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if true;
    }
  }
}
```

- ✗ Любой может читать данные
- ✗ Любой может изменять и удалять документы
- ✗ Риск утечки и злоупотреблений
- ✗ Подходит только для краткой учебной демонстрации



Нельзя оставлять в реальном проекте

Лучше

Ограничить доступ правилами

Firestore Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /products/{productId} {
      allow read: if true;
      allow write: if request.auth != null;
    }
  }
}
```

- ✓ Товары можно читать всем
- ✓ Изменять данные могут только вошедшие пользователи
- ✓ Доступ становится предсказуемым
- ✓ Хорошая база для перехода к production-правилам



Безопаснее и понятнее

Что запомнить

1

Тестовый режим — временно

2

В production доступ нужно ограничивать

3

Правила проверяют, кто и что может делать



Итог: сначала можно быстро запустить проект, но перед реальной работой обязательно замените `allow read, write: if true` на ограничивающие правила.

5. Как подключить Firebase к веб-приложению на JavaScript?

Три способа подключения:

- CDN (самый простой, для новичков)
- npm модули (для современных проектов)
- ES модули (самый современный)

CDN (Content Delivery Network) — сеть доставки контента. Позволяет подключать библиотеки прямо из интернета без скачивания.

Шаг 1: Подключение Firebase SDK

Подключаем Firebase SDK через CDN:

```
<script src="https://www.gstatic.com/firebasejs/12.8.0/firebase-app-compat.js"></script>  
<script src="https://www.gstatic.com/firebasejs/12.8.0/firebase-firestore-compat.js"></script>
```

Важно:

Используем -compat версии (совместимость)

Подключаем два скрипта: firebase-app и firebase-firestore

Код Firebase пишем после подключения SDK

Документация:

<https://firebase.google.com/docs/web/modular-upgrade?hl=ru>

Шаг 2: Конфигурация Firebase

Копируем из Firebase Console:

```
const firebaseConfig = {  
  apiKey: "AlzaSyD...",  
  authDomain: "ваш-проект.firebaseio.com",  
  projectId: "ваш-проект-id",  
  storageBucket: "ваш-проект.appspot.com",  
  messagingSenderId: "123456789012",  
  appId: "1:123456789012:web:abc123def456"  
};
```

Где взять:

- Firebase Console → Ваш проект
- Нажмите "Add app" → "Web"
- Скопируйте объект firebaseConfig

Шаг 3: Инициализация Firebase

Инициализируем Firebase приложение

```
const app = firebase.initializeApp(firebaseConfig);
```

Получаем доступ к Firestore

```
const db = firebase.firestore();  
console.log("✅ Firebase подключен!");
```

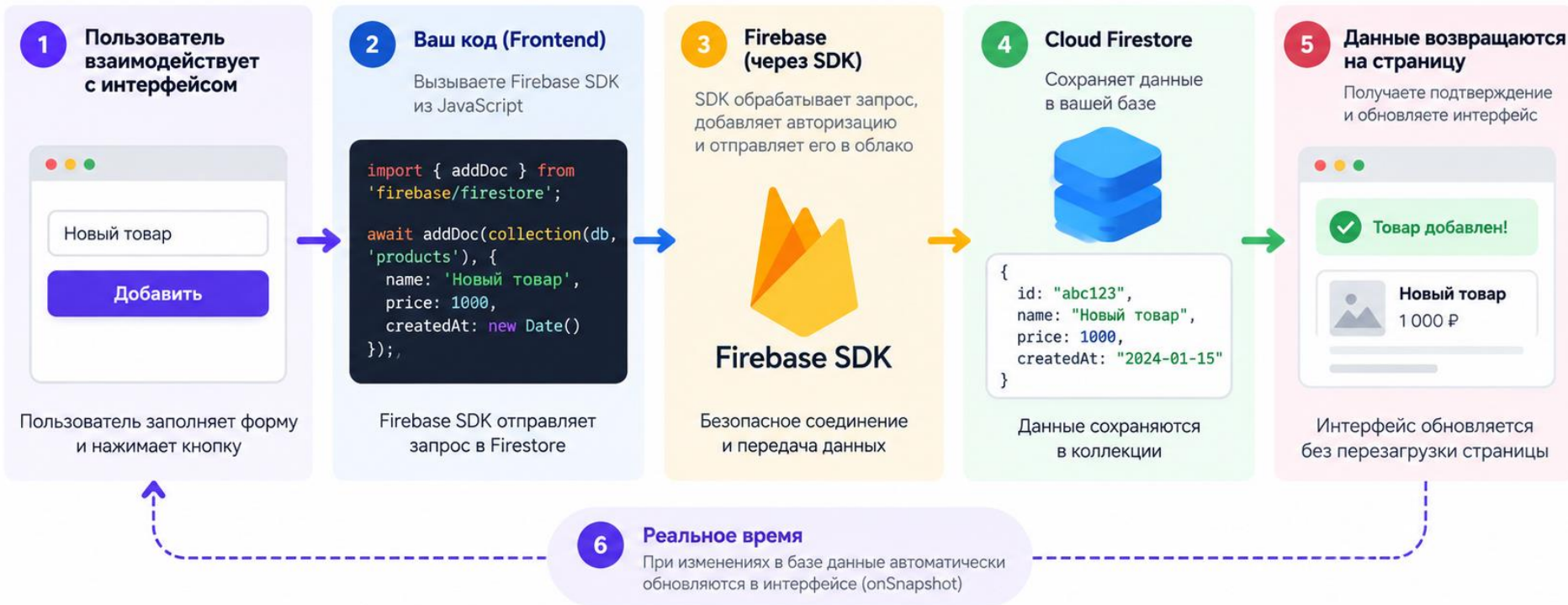
Что происходит:

- `firebase.initializeApp()` — создаёт приложение Firebase
 - `firebase.firestore()` — даёт доступ к базе данных
- `db` — это наш "мост" к Firestore

Путь данных: от интерфейса до Firestore и обратно

Как данные из вашего приложения попадают в базу и возвращаются обратно на страницу

Один запрос —
много возможностей



Итог: вы пишете обычный JavaScript, а Firebase берёт на себя всю сложную инфраструктуру — соединение, масштабирование, безопасность и синхронизацию данных в реальном времени.

Больше, чем база данных.
Это работает для вас.



Шаг 4: Выполнение запроса к Firestore

Метод `get()` для коллекции:

Получаем все документы из коллекции
'products'

```
const querySnapshot = await db.collection('products').get();
```

Что такое QuerySnapshot:

- Результат запроса к Firestore
- Содержит все найденные документы
- Имеет методы для работы с документами

Шаг 5: Работа с результатами

Преобразование документа:

```
querySnapshot.forEach((doc) => {  
  // doc.id — ID документа (автоматически сгенерированный)  
  // doc.data() — объект с данными документа  
  
  console.log("ID документа:", doc.id);  
  console.log("Данные документа:", doc.data());  
});
```

Вывод в консоль браузера:

```
// Простой вывод в консоль  
querySnapshot.forEach((doc) => {  
  const product = doc.data();  
  console.log(` ${product.name}: ${product.price} `);  
});
```

Вывод на HTML-страницу:

```
const container = document.getElementById('products');
```

```
querySnapshot.forEach((doc) => {  
  const product = doc.data();
```

```
  // Создаём HTML для каждого товара
```

```
  container.innerHTML += `
```

```
    <div class="product">
```

```
      <h3>${product.name}</h3>
```

```
      <p>Цена: $$${product.price}</p>
```

```
    </div>
```

```
  `;  
});
```

Пример:

```
<body>
  <h1>Товары из Firebase</h1>
  <div id="output"></div>
  <!-- Подключение Firebase SDK через CDN -->
  <script src="https://www.gstatic.com/firebasejs/12.8.0/firebase-app-compat.js"></script>
  <script src="https://www.gstatic.com/firebasejs/12.8.0/firebase-firestore-compat.js"></script>

  <script>
    // Конфигурация (замените на свою!)
    const firebaseConfig = {
      apiKey: "AIzaSyALEGp1m4WQVmjEJ7JWvK93cvSOFh4ARw",
      authDomain: "my-test-project-e0b7a.firebaseio.com",
      projectId: "my-test-project-e0b7a",
      storageBucket: "my-test-project-e0b7a.firebaseio.com",
      messagingSenderId: "962855943660",
      appId: "1:962855943660:web:278bfe37376e6b046154c9",
    };
  </script>
</body>
```

```
// Подключение
const app = firebase.initializeApp(firebaseConfig);
const db = firebase.firestore();

console.log("Firestore подключен!");
```

```
// Загрузка данных
db.collection("products")
  .get()
  .then((snapshot) => {
    const output = document.getElementById("output");
    snapshot.forEach((doc) => {
      const data = doc.data();
      output.innerHTML += `<p>${data.name}: ${data.price}</p>`;
    });
  })
  .catch((error) => {
    console.error("Ошибка:", error);
    document.getElementById("output").innerHTML = "Ошибка загрузки";
  });
</script>
</body>
```

← → ↻ ⓘ 127.0.0.1:5500/index.html

Товары из Firebase

iPhone 15: \$549000

AirPods Pro: \$99000

MacBook Pro: \$999000

ⓘ DevTools is now available

⌵ ⌵ Elements Console

⏮ ⏭ top ▼ 👁 🔍

Firestore подключен!

>

Вывод по лекции:

- Поняли Firebase — облачный бэкенд для фронтендеров
- Создали проект в Firebase Console
- Настроили Firestore (документную БД)
- Подключили к HTML через CDN
- Вывели данные из облака на страницу

Контрольные вопросы:

- Что такое Firebase и зачем он фронтендеру?
- Чем Firestore отличается от SQL-баз?
- Как получить конфигурацию для подключения Firebase?
- Какие шаги для подключения Firebase к HTML через CDN?
- Как получить все товары из коллекции 'products'?
- Опишите иерархию Firestore на примере магазина

хекслет колледж

@HEXLY.KZ